

Artificial Intelligence Assignment Report 4

Warren Roberts

Student ID: 100450116

Student Email: wlr006@uark.edu

University of Arkansas

0 Late Days Used

Abstract

This report covers my implementation for Assignment 4, which focuses on neural networks and backpropagation using PyTorch. The results show that the binary network successfully learned all four operators, the MNIST classifier performed well, and the custom backpropagation implementation worked as expected.

Introduction

This assignment focuses on understanding how neural networks learn through forward propagation, loss computation, and backpropagation. The work is divided into three parts:

- Binary classification using a small neural network,
- Handwritten digit classification using the MNIST dataset,
- Manual implementation of backpropagation components using custom PyTorch autograd functions.

The main objective is to understand both how neural networks perform classification tasks and how gradients are used to update parameters during training.

Binary Network

Model Design

The binary classification model used a simple fully connected neural network with:

- Input size: 2
- Hidden layer: 4 neurons
- Output size: 1
- ReLU activation in the hidden layer
- Sigmoid activation in the output layer

This design was sufficient to learn the AND, OR, XOR, and NOR operators. The hidden layer was especially important for XOR, since it is not linearly separable.

Training Setup

The binary network was trained with the following hyperparameters:

- Optimizer: Stochastic Gradient Descent (SGD)
- Learning rate: 0.1
- Momentum: 0.9
- Number of iterations: 1000 for AND, OR, and NOR
- Number of iterations: 2000 for XOR
- Loss function: Binary Cross Entropy Loss

Results

The model learned all four operators successfully. The final accuracies were:

- AND: 1.00
- OR: 1.00
- XOR: 1.00
- NOR: 1.00

The loss curves for each operator are shown below.

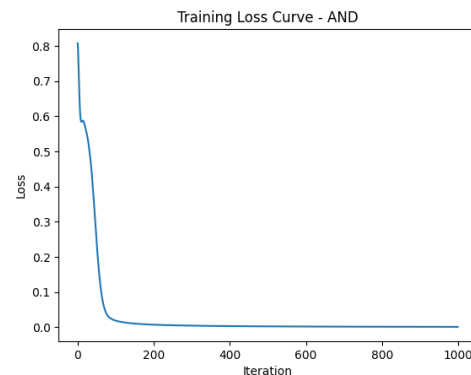


Figure 1: Training loss curve for the AND operator.

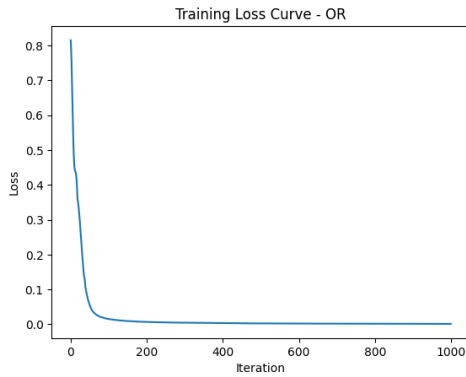


Figure 2: Training loss curve for the OR operator.

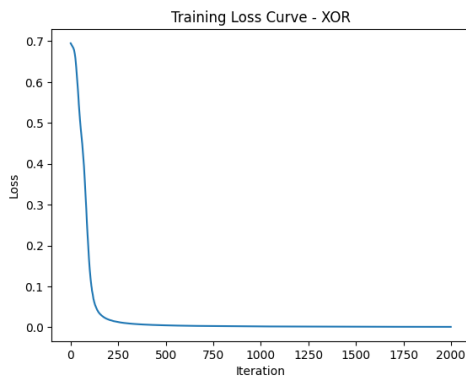


Figure 3: Training loss curve for the XOR operator.

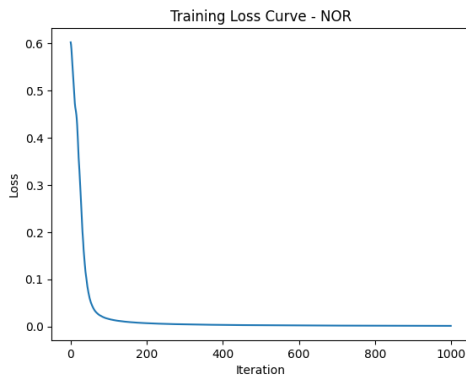


Figure 4: Training loss curve for the NOR operator.

The curves show rapid convergence for AND, OR, and NOR. XOR also converged successfully, although it required a nonlinear hidden layer and a slightly longer training process.

Digit Classification

Model Architecture

The digit classifier, `DigitNetwork`, is a fully connected neural network. Each MNIST image of size $1 \times 28 \times 28$ is flattened into a vector of length 784. The network consists of:

- Flatten layer
- Linear layer: $784 \rightarrow 128$
- ReLU activation
- Linear layer: $128 \rightarrow 64$
- ReLU activation
- Output layer: $64 \rightarrow 10$

Parameter Count

The number of trainable parameters in each layer is:

- First linear layer: $(784 \times 128) + 128 = 100,480$
- Second linear layer: $(128 \times 64) + 64 = 8,256$
- Output layer: $(64 \times 10) + 10 = 650$

Therefore, the total number of trainable parameters is:

$$100,480 + 8,256 + 650 = 109,386$$

Training Setup

The MNIST model was trained using:

- Batch size: 32
- Number of epochs: 1
- Learning rate: 0.1
- Optimizer: SGD with momentum 0.9
- Loss function: CrossEntropyLoss

Training Performance

The training loss consistently decreased over time, while the training accuracy increased steadily, showing that the model was learning effectively.

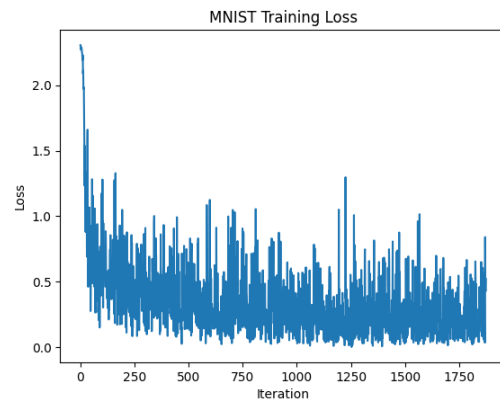


Figure 5: MNIST training loss over iterations.

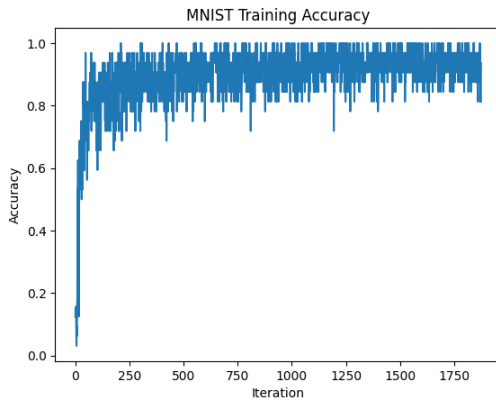


Figure 6: MNIST training accuracy over iterations.

The average iteration time during training was:

0.001365 seconds

The total training time was:

8.974171 seconds

Evaluation Results

The trained model was evaluated on the MNIST test set with batch size 1. The final test accuracy was:

Final Accuracy = 0.9435

This is a strong result for a simple fully connected network trained for only one epoch.

Confusion Matrix

The confusion matrix is shown below.

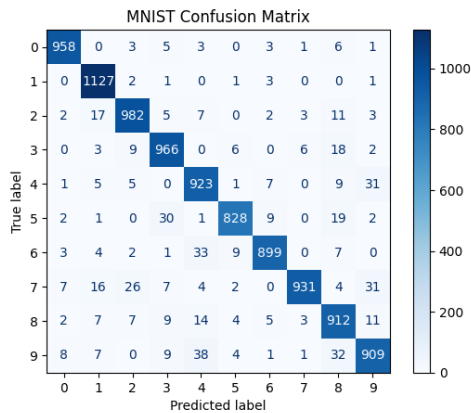


Figure 7: Confusion matrix for the MNIST test set.

The confusion matrix shows that most digits were classified correctly. Most errors occurred between visually similar digits, such as 4 and 6, 7 and 9, or 8 and 9.

Correct and Incorrect Predictions

Examples of correctly and incorrectly predicted samples are shown below. Each image lists the true label (T), predicted label (P), and confidence score (C).

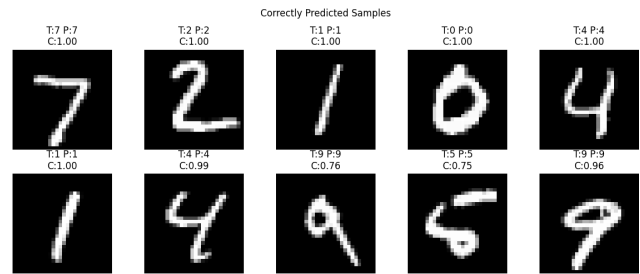


Figure 8: Examples of correctly predicted MNIST digits with confidence scores.

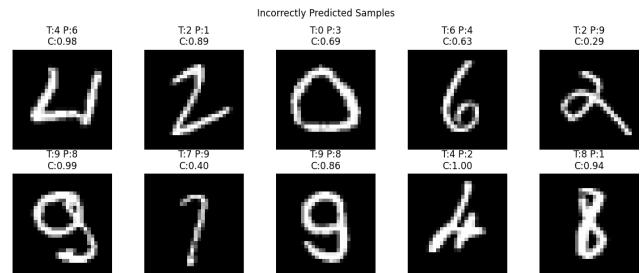


Figure 9: Examples of incorrectly predicted MNIST digits with confidence scores.

The correct examples generally have high confidence scores, while the incorrect examples often involve ambiguous handwriting or digits with shapes similar to other classes.

Backpropagation Implementation

Custom Sigmoid Function

The sigmoid activation function was implemented as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative for the backward pass was:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

In the forward pass, the sigmoid output was computed and stored for reuse during backpropagation. In the backward pass, the upstream gradient was multiplied by the sigmoid derivative.

Custom Linear Layer

The fully connected layer was implemented as:

$$y = xW + b$$

The gradients used in the backward pass were:

- Gradient with respect to input:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} W^T$$

- Gradient with respect to weights:

$$\frac{\partial L}{\partial W} = x^T \frac{\partial L}{\partial y}$$

- Gradient with respect to bias:

$$\frac{\partial L}{\partial b} = \sum \frac{\partial L}{\partial y}$$

Custom Network

A custom neural network, `MyLinearNetwork`, was built using:

- Two custom linear layers
- A custom sigmoid activation
- A softmax layer during evaluation

Results

The custom network was trained and evaluated using the MNIST framework. The final accuracy achieved by the custom backpropagation network should be reported here from the notebook results:

Final Accuracy = 0.95

This result confirms that the manually implemented forward and backward passes were correct and produced meaningful gradient updates. The custom backpropagation network achieved a final accuracy of 0.95, demonstrating that the implementation was successful.

Results Summary

- The binary network successfully learned AND, OR, XOR, and NOR with final accuracy of 1.00 on all four operators.
- The MNIST digit classifier achieved a final test accuracy of 0.9435.
- The MNIST model had an average iteration time of 0.001365 seconds and a total training time of 8.974171 seconds.
- The custom backpropagation network successfully trained using manually implemented forward and backward functions.

Conclusion

This assignment provided practical experience with neural networks and backpropagation at both a high level and a low level. The binary logic tasks demonstrated how small feed-forward networks can learn basic operators, including XOR when a hidden layer is used. The MNIST classifier showed that a simple fully connected model can achieve strong performance on handwritten digit classification. Finally, the manual implementation of sigmoid and linear layers reinforced how forward and backward passes work internally in neural networks.

Overall, this assignment strengthened my understanding of neural network training, gradient-based optimization, and the role of backpropagation in learning.